

VISUALIZACIÓN

PRÁCTICA 3: CALIDAD DE LA VISUALIZACIÓN. CHEKS

Yaiza Vicente Pérez

24 de febrero de 2026



Máster Universitario en Ciberseguridad e Inteligencia de Datos
Universidad de La Laguna

Índice

1. Enlace a Github	1
2. Análisis del script test_checks.py (paso 3)	1
3. Análisis del fichero definitions.py (paso 4)	1
4. Ejecución y verificación (paso 5)	1
5. Checks sin validar (paso 6)	1
6. Checks de mi proyecto (paso 7)	2
7. Verificación de fallos (paso 8)	5

1. Enlace a Github

<https://github.com/yaizavicenteperez/practica2-v/tree/cheks>

2. Análisis del script `test_checks.py` (paso 3)

El asset `islas_raw` carga el fichero `pwbi-1.csv` y lo devuelve como un `DataFrame` de `pandas`. Cuando se descomenten las líneas indicadas en el paso 6 añadirá una fila con datos ‘sucios’ que nos permitirá comprobar el funcionamiento del check cuando deba dar fallo.

El check `check_estandarizacion_islas` comprueba si los nombres de las islas están en un formato de capitalización uniforme. Es decir, que no existen duplicados causados por diferencias entre mayúsculas y minúsculas (ej.: `tenerife` y `Tenerife`). Esto lo hace comprobando cuántos valores únicos hay en el `DataFrame` antes y después de aplicar `capitalize()`. Si ve que ambos valores son iguales, el check pasa; si no, da fallo.

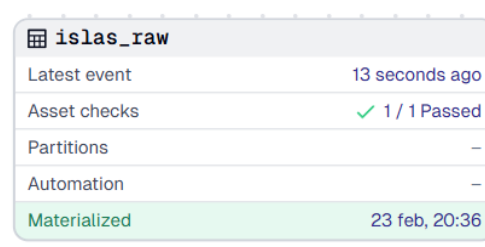
3. Análisis del fichero `definitions.py` (paso 4)

En el archivo `definitions.py` tenemos el objeto `Definitions` que es el que indica a Dagster qué assets y checks orquestar. En el manifiesto tenemos lo siguiente:

- El asset `islas_raw`, cargado desde el módulo `test_checks` utilizando `load_assets_from_modules`.
- El check `check_estandarizacion_islas`, cargado desde el módulo `test_checks` utilizando `load_asset_checks_from_modules`.

4. Ejecución y verificación (paso 5)

Como podemos ver en la figura 1 el check `check_estandarizacion_islas` se ejecuta automáticamente al materializar el asset `islas_raw` y pasa correctamente, lo que nos indica que todos los nombres de las islas en el dataset que estamos utilizando tienen una capitalización uniforme.



islas_raw	
Latest event	13 seconds ago
Asset checks	✓ 1 / 1 Passed
Partitions	—
Automation	—
Materialized	23 feb, 20:36

Figura 1: Comprobación de validación del check.

5. Checks sin validar (paso 6)

Al haber añadido una fila con ‘tenerife’ (en minúsculas), el tendremos más categorías únicas antes de aplicar `capitalize()` que después, por lo que el check fallará.

En la figura 2 podemos ver que el check no ha pasado y se ha puesto en rojo, y en la figura 3 podemos ver que, efectivamente, hay una discrepancia entre las categorías detectadas y las es-

peradas. Antes de descomentar las líneas (20:36) teníamos 2 categorías detectadas, mientras que después de descomentarlas (20:40) pasamos a tener 3.

islas_raw	
Latest event	12 seconds ago
Asset checks	✗ 0 / 1 Passed
Partitions	—
Automation	—
Materialized	23 feb, 20:40

Figura 2: Comprobación de fallo del check.

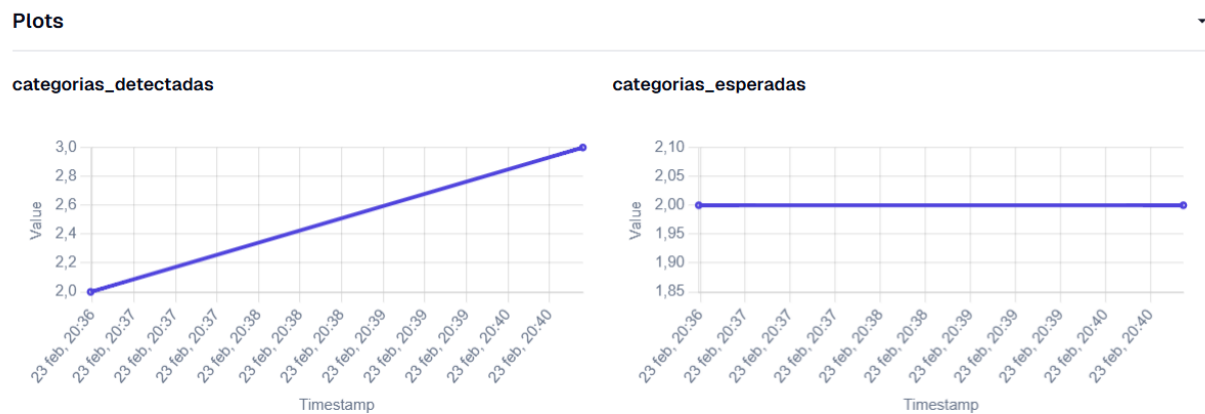


Figura 3: Categorías detectadas por el check vs categorías esperadas.

6. Checks de mi proyecto (paso 7)

Para mi proyecto de la entrega anterior se han implementado 9 checks, que se han implementado en tres etapas diferentes del pipeline como se muestra en la tabla 1. A continuación realizaremos una breve explicación de las tres etapas con sus checks:

- **Carga (raw).** Se han implementado siete checks:
 - **check_nulos_renta**, que actúa sobre el asset `raw_renta_data`, verifica que en la columna `OBS_VALUE` no existan valores nulos, puesto que estos podrían darnos problemas más tarde durante el análisis y la visualización de los datos.
 - **check_columnas_renta**, que actúa sobre el asset `raw_renta_data`, comprueba que el dataset de renta contiene todas las columnas esperadas antes de continuar con las transformaciones.
 - **check_n_tipos_ingreso**, que actúa sobre el asset `raw_renta_data`, verifica que existen 5 tipos de ingreso, que son los esperados para la gráfica.
 - **check_suma_porcentajes**, que actúa sobre el asset `raw_renta_data`, comprueba que los porcentajes por isla son correctos y suman aproximadamente 100 %
 - **check_n_categorias_color**, que actúa sobre el asset `raw_renta_data`, verifica que el número de tipos de ingreso no supera 8, que es el límite de categorías que vamos a poder poner en una gráfica para que se pueda representar correctamente. Se verifica en

esta etapa porque el número de tipos de ingreso es una propiedad de los datos originales, por lo que es preferible detectar cualquier problema antes de continuar con las transformaciones.

- `check_nulos_codislas`, que actúa sobre el asset `raw_codislas_data`, verifica que no haya nulos en dicho dataset.
 - `check_nulos_estudios`, que actúa sobre el asset `raw_estudios_data`, verifica que no haya nulos en las columnas clave del dataset correspondiente.
- **Transformación.** En esta etapa se han implementado los siguientes tres checks:
- `check_n_islas`, que actúa sobre el asset `islas_agregadas_data` y que verifica que existan exactamente 7 islas en el DataFrame.
 - `check_continuidad_temporal`, que actúa sobre el asset `datos_evolucion_temporal` y comprueba que están presentes todos los años del rango 2015-2023, para evitar saltos en las gráficas de evolución temporal.
 - `check_municipios_tenerife`, que actúa sobre `datos_heatmap_tenerife` para comprobar que, tras el filtrado por isla, nos hemos quedado con los 31 municipios de Tenerife.
- **Visualización.** Se ha implementado el siguiente check, que actúa sobre el asset `viz_estructura_isla_2023`:
- `check_archivo_png_generado` que verifica que el archivo PNG se haya generado correctamente y que no esté vacío. Este check se ha puesto solo en esta gráfica por simplificar, pero podría añadirse para todas las gráficas que se generan.

Etap	Nombre	Descripción	Lógica	Gestalt	Metadata
Carga (Raw)	check_nulos_renta	Detecta nulos en OBS_VALUE	<code>isna().sum() == 0</code>	Figura y Fondo	pct_nulos, filas_afectadas
Carga (Raw)	check_columnas_renta	Verifica que existen las columnas esperadas	<code>cols.issubset(df.columns)</code>	Continuidad	columnas_faltantes
Carga (Raw)	check_n_tipos_ingreso	Exactamente 5 tipos de ingreso	<code>nunique() == 5</code>	Carga cognitiva	n_tipos_detectados
Carga (Raw)	check_suma_porcentajes	Porcentajes por isla suman $\approx 100\%$	<code>groupby().sum() ≈ 100</code>	Proporcionalidad	islas_incorrectas, suma_min, suma_max
Carga (Raw)	check_n_categorias_color	Máximo 8 categorías para paleta cualitativa	<code>nunique() <= 8</code>	Similitud	n_categorias
Carga (Raw)	check_nulos_codislas	Detecta nulos en columnas clave de codislas	<code>isna().sum() == 0</code>	Figura y Fondo	filas_afectadas
Carga (Raw)	check_nulos_estudios	Detecta nulos en columnas clave de estudios	<code>isna().sum() == 0</code>	Figura y Fondo	filas_afectadas
Transf.	check_n_islas	Exactamente 7 islas en el dataset	<code>nunique() == 7</code>	Similitud	n_islas_detectadas
Transf.	check_continuidad_temporal	Años completos de 2015 a 2023	<code>set(range(2015, 2024))</code>	Continuidad	años_faltantes, rango_temporal
Transf.	check_municipios_tenerife	31 municipios en Tenerife	<code>nunique() == 31</code>	Proximidad	n_municipios
Visualiz.	check_archivo_png_generado	El fichero PNG existe y no está vacío	<code>exists() and size > 0</code>	Veracidad Visual	archivo, existe, tamaño_bytes

Tabla 1: Checks de calidad del pipeline de visualización de rentas en Canarias

Todos los checks se validan con los datos originales con los que estamos trabajando como podemos ver en la figura 4.

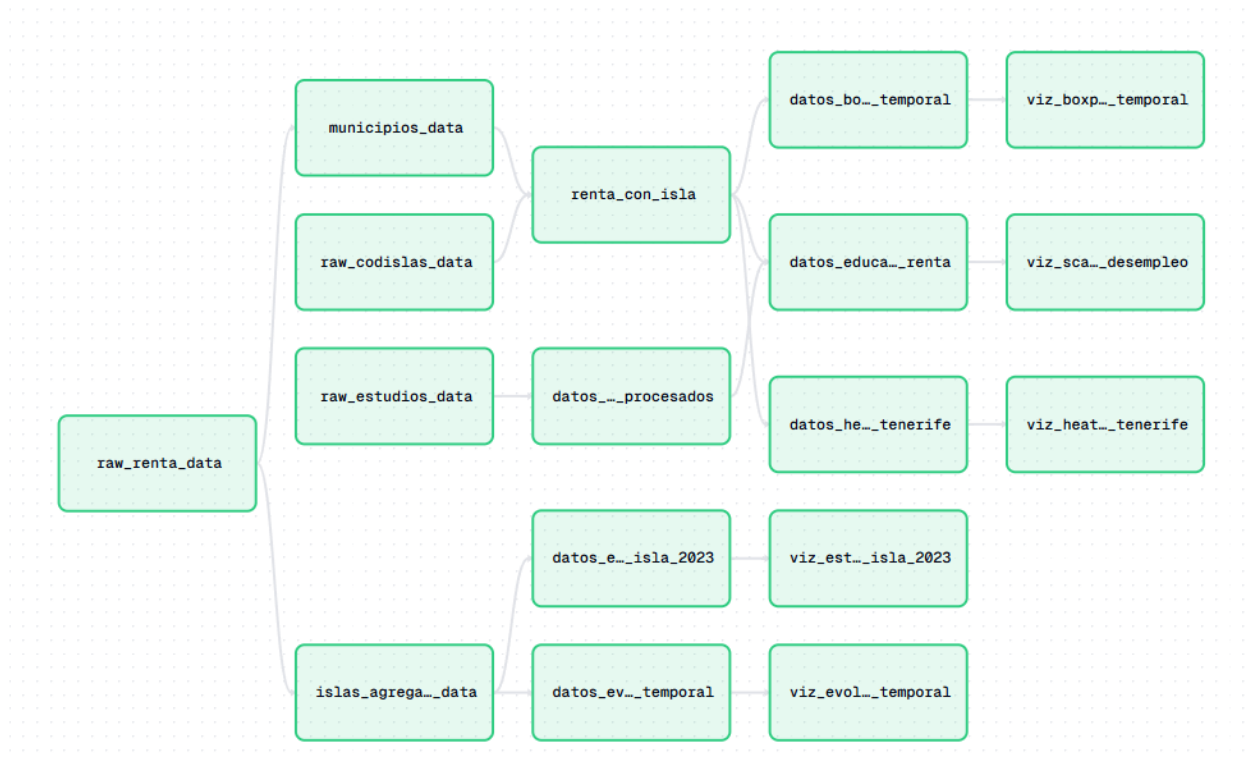


Figura 4: Visualización de los assets materializados con todos los checks validados.

7. Verificación de fallos (paso 8)

Para comprobar si los checks están funcionando correctamente vamos a introducir casos que los incumplan y volvemos a materializar los assets. Para ello hemos introducido en el archivo *assets_file.py* la variable `FORZAR_FALLOS`. Cuando esta variable toma el valor `FALSE` el programa se ejecuta de manera normal, pero cuando toma el valor `TRUE` se introducen casos que incumplen los checks en todos los assets que tienen algún check asociado

En la figura 5 podemos ver como ahora han fallado los checks, lo que nos indica que están funcionando correctamente. Además de los assets con check aparecen en rojo algunos más, este se debe a que una de las modificaciones que hemos añadido para que se fallaran los checks implicaba la eliminación `TERRITORIO.CODE`, lo que produce un fallo en cascada desde el asset `municipios_data`, en el que intentamos utilizar dicha columna.

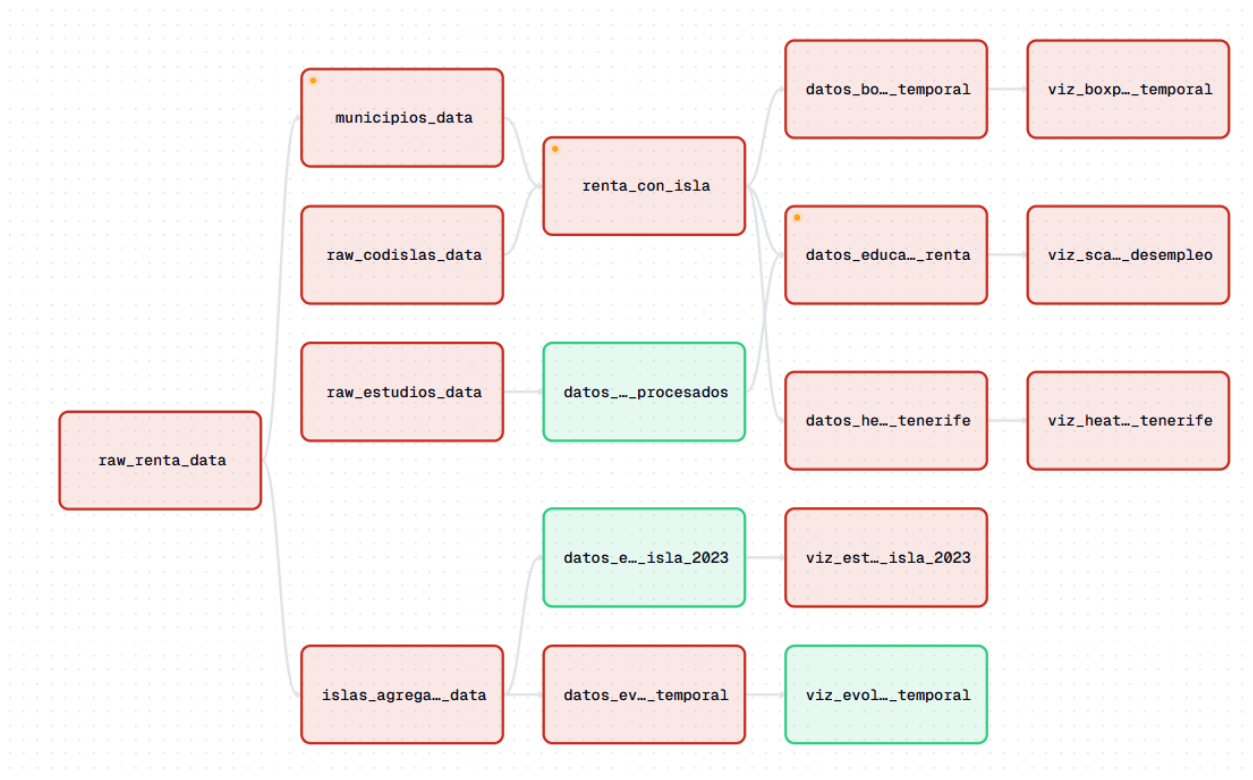


Figura 5: Visualización de los assets materializados con todos los checks fallados.